

Explanation of Deterministic and Probabilistic Health-Economic Markov Model Code

Background and context

Health-economic evaluations use **cost-effectiveness analysis (CEA)** to compare mutually exclusive strategies on both costs and health outcomes. The goal is to identify the strategy that provides the greatest benefit at an acceptable level of efficiency. A CEA orders strategies by increasing cost, removes those that are dominated, and then computes incremental cost-effectiveness ratios (ICERs) for the non-dominated strategies[1]. Incremental costs and outcomes allow the decision maker to determine whether the cost per unit of benefit is acceptable. The dampack R package implements many of the calculations and visualizations required for CEA, including deterministic and probabilistic sensitivity analyses.

The uploaded code implements a **three-state semi-Markov model** to evaluate the cost-effectiveness of adding a novel therapy (bevacizumab) to standard-of-care chemotherapy in metastatic colorectal cancer. The model distinguishes three health states—progression-free survival (PFS), progressed disease (PD) and death. The code is written as an R function (`oncologySemiMarkov`) and is embedded within a larger R Markdown workflow (`Markov_3state.txt`), which performs deterministic and probabilistic analyses and creates output tables. The function structure and use of dampack mirrors the examples provided in the package vignettes for deterministic sensitivity analysis[2] and probabilistic sensitivity analysis[3].

A **cohort state-transition model (cSTM)** is a discrete-time Markov model in which a homogeneous cohort moves among mutually exclusive and collectively exhaustive health states over time. The cSTM is defined by a state vector that stores the distribution of the cohort across states, a transition probability matrix whose rows sum to one, and a trace that records the proportion of the cohort in each state over time. In a Markov model the **transition probabilities depend only on the current state**, not on prior history—this is the Markovian assumption. The uploaded code uses a semi-Markov approach because the transition probabilities between states depend on time since entering PFS (derived from survival curves) rather than remaining constant.

Definition of the model function (`oncologySemiMarkov`)

The `oncologySemiMarkov` function accepts a list of parameters (`l_params_all`), including transition parameters, costs and utilities. Within the function, parameters are unpacked via `with(as.list(l_params_all), ...)`. The key steps are:

1. Setting up cycle times

- The model uses a half-month cycle length ($t_{\text{cycle}} = 0.25$ years) and runs for n_{cycle} cycles. A vector of time points t is created using `seq(from = 0, by =  $t_{\text{cycle}}$ , length.out =  $n_{\text{cycle}} + 1$ )`.

2. Time-dependent transition probabilities

The main innovation of this model is **time-dependent transitions from PFS to progression (PD) or death**. Instead of assuming constant probabilities each cycle, the function constructs a **survival function** for the PFS state based on parametric survival models. For standard-of-care (SoC), Weibull parameters (`coef_weibull_shape_SoC` and `coef_weibull_scale_SoC`) are used to compute the probability of remaining progression-free at each time point via `pweibull(q = t, shape =  $\exp(\text{coef\_weibull\_shape\_SoC})$ , scale =  $\exp(\text{coef\_weibull\_scale\_SoC})$ , lower.tail = FALSE)`. The code comments emphasise that the coefficients returned by the `flexsurv` package are log-transformed and need to be exponentiated before use.

To facilitate sensitivity analyses, the function introduces a **hazard ratio for SoC** (`HR_FP_SoC`), which is initially set to 1. The survival function is converted to a hazard function using $H(t) = -\ln(S(t))$; multiplying by `HR_FP_SoC` allows the model to scale the hazard for SoC if required. The hazard is then converted back to a survival function ($S(t) = \exp(-H(t))$). This mechanism allows the one-way or two-way deterministic sensitivity analysis to vary the baseline event rate by applying a hazard ratio to SoC without altering the underlying distribution.

For the experimental strategy, a separate hazard ratio (`HR_FP_Exp`) is applied to the SoC hazard to obtain the experimental hazard ($H_{\text{Exp}}(t) = H_{\text{SoC}}(t) \times \text{HR}_{\text{FP_Exp}}$), which is then exponentiated to produce the survival function for the experimental treatment. By converting survival functions to hazard functions and back, the model maintains the correct proportional hazard relationship across all cycles.

3. Converting survival to cycle-specific transition probabilities

Given a survival function $S(t)$ at discrete time points, the probability of leaving the PFS state in cycle i is computed as

$$p_i = 1 - \frac{S(t_{i+1})}{S(t_i)},$$

which reflects the **conditional probability** of transition in a discrete-time cSTM. The function creates vectors `p_PFS0S_SoC` and `p_PFS0S_Exp` for the probabilities of progressing from PFS to PD or death under each strategy. The comment illustrates the calculation: if the survival in PFS drops from 1.0 at time 0 to 0.9948 at 0.5 cycles, the probability of leaving PFS in that cycle is $1 - 0.9948 / 1.0 = 0.0052$.

After computing the probability of death directly from PFS (using a beta-distributed parameter P_OSD_SoC for the SoC death hazard and applying hazard ratios for the experimental strategy), the PFS-to-progression probability is reduced accordingly to avoid double-counting deaths.

4. Transition probabilities for progression and death

Once in the progressed state (PD), patients face a constant probability of dying (p_OSD_SoC) per cycle. The code documents the reasoning behind using a fixed probability: the survival curves only describe first-line treatment, so after progression a simpler deterministic transition probability suffices. Hazard ratios can again be applied to vary the death rate in sensitivity analyses.

5. Markov trace, costs and utilities

The function constructs a **transition probability matrix** m_P for each strategy. The matrix rows correspond to the three states (PFS, PD, death) and columns indicate the probability of moving to each state in the next cycle. For example, the probability of remaining in PFS under SoC is $1 - (p_PFSOS_SoC[i] + p_FP_death_SoC[i])$, the probability of progressing to PD is $p_PFSOS_SoC[i]$, and the probability of dying directly from PFS is $p_FP_death_SoC[i]$. Probabilities from PD to death and from death to death are specified similarly. At each cycle, the cohort distribution is updated by multiplying the current state vector by the transition matrix, a standard operation in cSTMs.

Costs and QALYs are assigned to each state and multiplied by the cohort distribution to obtain the expected cost and utility in each cycle. The model uses half-cycle correction and applies discount factors. Total discounted costs and QALYs are calculated by weighting the per-cycle costs and utilities by discount weights and summing across cycles[4]. The function returns a data frame with strategy names, total costs, total QALYs and net monetary benefits (NMB) for a specified willingness-to-pay threshold.

Deterministic analysis and sensitivity analyses

The R Markdown file `Markov_3state.txt` sets up the deterministic evaluation and sensitivity analyses. It loads necessary packages (`dampack`, `heemod`, `flexsurv`, etc.), defines base-case parameters (costs, utilities, hazard ratios, treatment costs, adverse events, discount rates) and creates a wrapper around `oncoSemiMarkov` that supplies strategy-specific parameter values. Using this wrapper, the script obtains base-case costs and QALYs for standard-of-care and the experimental strategy and calculates the incremental cost-effectiveness ratio using `dampack::calculate_icers()`[5].

One-way and two-way deterministic sensitivity analysis (DSA)

The `dsa_generation` vignette defines DSA as running the model over a grid of values for the parameter(s) of interest while holding other parameters fixed[6]. The script uses `create_dsa_oneway()` to specify the range (minimum and maximum) of each parameter

and `run_owsa_det()` to run the model across that range. For example, hazard ratios for progression (HR_FP_Exp) or death (HR_PD_Exp) are varied $\pm 20\%$ to examine how the ICER changes. The output can be visualized with `owsa_tornado()`, which produces a tornado diagram ranking parameters by their impact on the NMB.

The code also sets up a two-way sensitivity analysis (TWSA) by specifying a grid over two parameters. In the `oncologySemiMarkov` function, hazard ratios for SoC and the experimental treatment are applied sequentially so that changes in the baseline event rate and the treatment effect covary logically—this ensures that the hazard ratio always reflects a proportional change relative to the current base event rate, as described in the comments.

Probabilistic sensitivity analysis (PSA)

Deterministic analyses vary parameters one or two at a time, but they do not capture joint uncertainty across all parameters. **Probabilistic sensitivity analysis (PSA)** addresses this by drawing parameter values from probability distributions and running the model many times. The `psa_generation` vignette notes that PSA translates parameter uncertainty into decision uncertainty by repeatedly sampling from parameter distributions and calculating outcomes[3]. Common distributions include normal, beta, gamma and log-normal, and each PSA sample requires one draw from each distribution.

The script uses `gen_psa_samp()` from `dampack` to sample from the specified distributions. For instance, costs per cycle might be drawn from gamma distributions and probabilities from beta distributions; hazard ratios can be sampled from log-normal distributions. The sample is stored in a data frame where each row corresponds to one simulation. The `run_psa()` function applies the wrapper model to every sample to compute costs, QALYs and NMB for both strategies. The resulting `psa` object contains matrices of cost and effectiveness outcomes and the sampled parameter values.

PSA analysis

Once the `psa` object is created, the script calculates cost-effectiveness acceptability curves (CEACs) using `ceac()`. A CEAC plots the probability that each strategy is cost-effective against different willingness-to-pay (WTP) thresholds; the underlying net monetary benefit is $\text{NMB} = \text{WTP} \times \text{Effectiveness} - \text{Cost}$ [7]. The script may also compute expected value of perfect information (EVPI) and expected value of perfect parameter information (EVPPi) using `dampack`'s `calc_evpi()` and `calc_evppi()` functions (not shown in the provided code but available in the package). These metrics quantify the value of reducing parameter uncertainty.

Creating parameter table

The auxiliary script `Creating Table 1.txt` produces a Word table summarizing the base-case value, minimum and maximum values and assumed distribution for each parameter. It uses the `flextable` and `officer` packages to create a formatted table with

headings such as **Cost per cycle**, **Adverse event incidence**, **Utility**, **Hazard ratios** and **Discount rate**. The code references distribution functions (gamma, beta, log-normal) that were used to generate PSA samples. This table aids readers in understanding the parameter inputs and their uncertainty ranges.

Relationship to dampack vignettes and manual

The structure of the uploaded code closely follows the modelling framework promoted in the dampack vignettes. The vignettes emphasize that the user-defined model must accept a list of parameters and return a data frame of outcomes for each strategy[8]. They also describe constructing a wrapper function to simulate multiple strategies, which is exactly how `Markov_3state.txt` wraps the `oncoSemiMarkov` function and computes outcomes for standard-of-care and the experimental strategy. The manual explains the functions used to calculate ICERs (`calculate_icers()`), generate deterministic sensitivity analyses (`create_dsa_oneway()` and `owsa()`), generate PSA samples (`gen_psa_samp()`), and create CEACs (`ceac()`). The code uses these functions as documented.

The detailed comments within `oncoSemiMarkov` highlight issues encountered in modelling time-dependent transitions (e.g., how to combine hazard ratios with survival curves) and align with the methods described in the state-transition modelling tutorial. In particular, the code takes care to convert between rates and probabilities when cycle lengths differ, as recommended in the tutorial.

Summary

The uploaded R code implements a semi-Markov cohort state-transition model to evaluate the cost-effectiveness of adding bevacizumab to chemotherapy for metastatic colorectal cancer. The model defines progression-free, progressed and death health states; uses parametric survival functions and hazard ratios to obtain time-dependent transition probabilities; and calculates costs and QALYs for each strategy. Deterministic analyses compare base-case outcomes and perform one-way and two-way sensitivity analyses using dampack's DSA functions, while probabilistic analyses sample parameters from distributions and evaluate uncertainty using CEACs and other outputs. The modelling approach and use of dampack functions are consistent with the package's vignettes and the general principles of cohort state-transition modelling.

[1] [5] Basic Cost Effectiveness Analysis

https://cran.r-project.org/web/packages/dampack/vignettes/basic_cea.html

[2] [4] [6] [8] Deterministic Sensitivity Analysis: Generation

https://cran.r-project.org/web/packages/dampack/vignettes/dsa_generation.html

[3] [7] Probabilistic Sensitivity Analysis: Analysis

https://cran.r-project.org/web/packages/dampack/vignettes/psa_analysis.html